Patient-Specific Multi-Hop Drug Interaction Risk Discovery

A comparative database-systems study of pairwise vs. graph-native reasoning for polypharmacy risk explanation

Prepared by: V Saatwik | Team of 2 (CSE, PES University) | Course: Database Management Systems

In one line: We are not predicting new drug interactions — that is an active, well-resourced ML research field. We are building a database system that takes a real patient's *full* medication list and traces documented, multi-hop biological pathways (drug → enzyme → drug → adverse effect) to surface and *explain* risks that pairwise interaction checking misses.

1. Problem Statement

Patients on multiple concurrent medications (polypharmacy) face interaction risks that go beyond simple drug-A-vs-drug-B checking. Two drugs that show no direct listed interaction can still create a real risk indirectly — for example, both being metabolized by the same liver enzyme, or both elevating the same adverse effect through different biological routes. Conventional interaction checkers (commercial tools such as Lexicomp/Micromedex, and most hospital EHR alert systems) only evaluate drug pairs in isolation and give little to no explanation of *why* a risk was flagged, which contributes to a well-documented problem called **alert fatigue** — clinicians ignoring warnings because too many are shallow or unexplained.

This project builds a system that reasons over a patient's *entire* current medication list at once, using a graph of documented pharmacological relationships, and returns a traceable path for every flagged risk — not just a severity label.

2. Why This Is Not Already Solved

A literature check was deliberately done before committing to this idea, specifically to avoid pitching something that already exists. The knowledge-graph-based drug-interaction space is active and well-funded — but everything found in it solves a *different* problem than this project.

System	What it actually does	Why it is different from this project
MolecBioNet (ACM KDD 2025)	Trains a graph neural network to **predict the interaction type** between **one drug pair** at a time, evaluated by classification accuracy on benchmark datasets (Ryu's dataset, DrugBank).	It is a machine-learning modeling paper with no patient data, no prescriptions, no application layer, and no notion of a patient's full drug list. It predicts unknown pairwise interactions; we only traverse known relationships.
KnowDDI (Nature Communications Medicine, 2024)	Similar pairwise DDI-type prediction using knowledge-subgraph learning for interpretability.	Same gap as above — pairwise, benchmark-driven, no polypharmacy or patient-record modeling, no deployed decision-support workflow.

System	What it actually does	Why it is different from this project
Commercial checkers (Lexicomp, Micromedex, hospital EHR alerts)	Pairwise lookup against a static interaction database; binary/severity flag only.	No multi-hop reasoning, no path-level explanation, and proprietary/closed — not something we can inspect, extend, or benchmark against directly.

The gap this project targets: *nobody in the reviewed literature reasons over a patient's whole concurrent medication list as a graph-traversal problem, and nobody pairs that with a proper transactional patient-record system. This project's contribution is a* ***systems/database contribution*** *(how to model, store, query, and explain this efficiently across two database paradigms) rather than an ML-modeling contribution — which keeps it honest, scoped, and achievable in one semester while still being a real, defensible claim of novelty.*

3. Data — What We Use and Why

No data is invented at the level that matters for validity: the *relationships* in the knowledge graph are sourced from real, citable pharmacological databases. Only the *patients* (who is taking what) are synthetic, which is standard and defensible for a systems evaluation, since the object under test is the traversal system, not a claim about real individuals.

Data component	Source	Used for
Drug entities, drug–drug interactions, severity, and mechanism annotations (e.g. CYP450-enzyme-mediated interactions where documented)	**DrugBank** academic/education dataset (free for student use) — the standard, citable reference dataset used across DDI literature, including the papers reviewed above.	Backbone of the Neo4j knowledge graph: Drug, Protein/Enzyme, Condition, Adverse Event nodes and their documented relationships.
Additional adverse-event / side-effect associations	**FAERS** (FDA Adverse Event Reporting System) — public, downloadable, used by prior DDI-surveillance literature.	Enriches the AdverseEvent side of the graph beyond DrugBank's own annotations.
Patients, prescriptions, dosages, conditions, encounters	**Synthetic**, generated to control complexity: 1,000–5,000 patients, 2–8 concurrent medications each, drawn from the DrugBank drug pool.	Relational core (PostgreSQL). Synthetic here is standard practice — we are testing the traversal/explanation system, not making claims about real patients.
Doctors, roles, prescribing workflow	Synthetic, generated for the application layer (minimum 2 roles: Doctor, Admin).	Drives the Next.js application's authentication/authorization and prescribing flow.

Scope guardrails (kept deliberately tight so the project stays finishable in a semester):

- 500–2,000 drugs, 100–300 conditions, a few thousand documented relationships in the graph
- Maximum traversal depth of 3 hops — no unbounded graph search
- 1,000–5,000 synthetic patients, 2–8 concurrent medications each
- Only three risk types modeled: drug–drug, drug–condition, and adverse-event pathway risk

4. What We Do With the Data

4.1 Knowledge graph construction (Neo4j)

DrugBank and FAERS records are transformed into a labelled property graph with node types `Drug`, `Protein`, `Enzyme`, `Condition`, `AdverseEvent`, and relationship types such as `TARGETS`, `INHIBITS`, `CAUSES`, `INCREASES_RISK_OF`, and `TREATS`, each carrying properties such as severity and evidence source.

4.2 Patient-specific traversal

When a doctor views or updates a patient's active medication list, the application issues a bounded-depth Cypher traversal across that patient's current drugs, looking for paths connecting them through shared enzymes, proteins, or adverse-event nodes:

```
MATCH path = (d1:Drug)-[*1..3]-(d2:Drug)
WHERE d1.drugbank_id IN $activePatientDrugIds
  AND d2.drugbank_id IN $activePatientDrugIds AND d1 <> d2
RETURN path, [r IN relationships(path) | type(r)] AS pathway, length(path) AS hops
```

Every result carries the full path (not just an endpoint score), which becomes the explanation shown to the clinician — e.g. Warfarin → inhibits → CYP2C9 → metabolizes → Drug X → increases → bleeding risk.

4.3 Transactional core (PostgreSQL)

Patients, doctors, prescriptions, dosages, and conditions live relationally, with the properties this data genuinely needs: foreign-key integrity (no prescription for a nonexistent patient), constraint checks (dosage > 0, no duplicate active prescription), and transactional safety (a new prescription and its resulting risk-check/alert are written atomically). A trigger on prescription insert calls the traversal check and writes an `alert` row when a path is found.

4.4 Application layer (Next.js)

- **Doctor view:** patient chart → current medications → "Analyze Risk" → ranked risks, each expandable into its explanation path
- **Admin view:** graph/data management, alert audit log across all patients
- Role-based authentication (minimum 2 roles satisfies the course requirement); credentials via environment variables, never hard-coded; passwords hashed, not stored in plain text

5. Validation — How We Know It Works

This is the part that turns the project from a demo into an evaluated system. The core risk in the original idea was: "multi-hop finds more risks" is meaningless if the graph itself is invented — of course a deeper search finds more hits in a graph you built yourself. The fix is to ground the evaluation in DrugBank's own documented mechanism data wherever possible, and to add a small human-judgement check where it isn't.

Experiment 1 — Pairwise vs. multi-hop coverage

For the same set of synthetic polypharmacy patients, run (a) a conventional pairwise interaction check and (b) 2-hop and 3-hop graph traversal. Report how many additional risk pathways multi-hop surfaces, and — critically — how many of those additional pathways correspond to mechanisms DrugBank itself documents (e.g. a shared CYP450 enzyme), versus how many are structurally present in the graph but not independently corroborated. This turns "we found more" into "we found more, and here is how many are grounded in real documented mechanisms."

Experiment 2 — Graph-depth sensitivity

Measure how many *relevant* pathways appear at 1, 2, and 3 hops, and where the results start becoming noisy (spurious long-distance connections that share no real clinical relevance). This answers a genuine open question: at what traversal depth does additional reach stop paying for itself.

Experiment 3 — Query performance at scale

Benchmark traversal latency and relational query latency as the graph grows (1K → 10K → 50K → 100K nodes) and as the number of concurrent patient medications grows (2 → 8 drugs). This is a genuine systems result directly tied to the course — indexing strategy, query planning, and the relational-vs-graph performance tradeoff — and needs no clinical judgement to be valid.

Experiment 4 — Lightweight expert-adjacent check (stretch goal)

If time allows: take 30–50 detected multi-hop pathways and have a pharmacology-literate reviewer (e.g. a faculty member, or published case reports as a proxy) rate them as clinically sensible or not. This is the closest the project gets to "ground truth" beyond DrugBank's own annotations, and is explicitly framed as a stretch goal, not a blocking requirement.

What we will NOT claim

- We will not claim to discover new, previously unknown drug interactions
- We will not claim to outperform ML prediction models (MolecBioNet, KnowDDI) — different task, not a competition
- We will not present synthetic-patient results as clinical findings — they validate the system, not real-world outcomes

6. Technology Stack

Layer	Technology	Why
Frontend	Next.js	Required by the course; server-side rendering suits a data-heavy clinical dashboard.

Layer	Technology	Why
Backend / API	Next.js API routes (Route Handlers)	Keeps the stack unified; handles auth, validation, and orchestrates calls to both databases.
Relational database	PostgreSQL	ACID guarantees for patients/prescriptions; constraints, triggers, transactions — exactly what Units 1–2 of the syllabus cover, applied to data that genuinely needs them.
Graph database	Neo4j (Cypher)	Interaction pathways are inherently a network. Expressing 2–3 hop traversal relationally would need repeated self-joins that become unreadable and slow past one hop; Cypher expresses it natively.
Auth	NextAuth.js / JWT + bcrypt password hashing	Role-based access (Doctor, Admin), credentials via environment variables, passwords never stored in plain text.
Version control	Git + GitHub	Individual commit history to demonstrate both members' contributions, per course requirement.

7. Database Design — Snapshot

7.1 Relational schema (PostgreSQL) — core entities

Table	Key columns	Notes
patients	patient_id PK, name, dob, gender	—
doctors	doctor_id PK, name, specialization, role	Role column drives RBAC
drugs	drug_id PK, name, drugbank_id UNIQUE	Mirrors Neo4j Drug nodes for FK integrity
conditions	condition_id PK, name, icd_code	—
prescriptions	prescription_id PK, patient_id FK, doctor_id FK, drug_id FK, dosage, start_date, end_date, status	CHECK(dosage > 0); UNIQUE(patient_id, drug_id, status='active') prevents duplicate active prescriptions
alerts	alert_id PK, patient_id FK, pathway_json, severity, created_at, acknowledged_by FK	Written transactionally alongside the triggering prescription insert

7.2 Graph data model (Neo4j) — core entities

Node / Relationship	Properties	Purpose
(:Drug)	drugbank_id, name, category	Mirrors relational drugs table
(:Protein) / (:Enzyme)	name, uniprot_id	Mechanistic intermediates (e.g. CYP450 family)
(:Condition)	name, icd_code	Links to patient conditions where relevant
(:AdverseEvent)	name, faers_code	Terminal nodes for risk pathways
[:INTERACTS_WITH]	severity, mechanism, evidence_source	Direct documented DDI edges (DrugBank)
[:INHIBITS] / [:TARGETS]	evidence_source	Drug–enzyme/protein mechanism edges
[:CAUSES] / [:INCREASES_RISK_OF]	evidence_source	Pathway edges toward adverse events

Why two paradigms, stated explicitly (for the rubric's "justify your database choice" requirement): *PostgreSQL answers "who is taking what, and is that record valid and consistent?" — a transactional, constraint-heavy question. Neo4j answers "how are these biomedical entities connected, and what paths exist between them?" — a traversal question that gets combinatorially painful in SQL past a single join depth. Using one database for both would force either fragile multi-way self-joins (SQL for the graph question) or no transactional integrity (graph DB for the patient-record question).*

8. Advanced Features — Rubric Coverage

Course requirement	How this project satisfies it
≥ 4 advanced DB concepts	Transactions (prescription + alert written atomically), Triggers (auto risk-check on new prescription), Stored procedures/functions (risk scoring, path ranking), Indexing (patient_id/drug_id for alert queries; graph-side indexes on drugbank_id), plus graph traversal itself as the NoSQL-paradigm advanced feature.

Course requirement	How this project satisfies it
Two database paradigms, justified	PostgreSQL (transactional/relational) + Neo4j (graph) — justification in Section 7.2 above.
ER diagram + relational schema	Section 7.1, to be finalized as a formal ER diagram before implementation begins.
NoSQL/graph data model documentation	Section 7.2 — node/relationship types, properties, and retrieval strategy (Cypher example, Section 4.2).
Authentication & role-based access (≥ 2 roles)	Doctor and Admin roles with distinct views and permissions (Section 4.4).
Data validation & integrity	CHECK constraints, FK integrity, UNIQUE constraints against duplicate active prescriptions (Section 7.1).
Security	Environment-variable credentials, bcrypt-hashed passwords, no plaintext secrets in source.
Beyond CRUD / meaningful real-world problem	Core value is the multi-hop traversal and explanation, not create/read/update/delete on records — addressed directly in Sections 1–2.

9. Indicative Timeline

Phase	Duration	Deliverables
1. Data acquisition & scoping	Week 1–2	DrugBank/FAERS export cleaned and bounded (drug/condition/relationship counts fixed); ER diagram and graph schema drafted and approved by instructor.
2. Database implementation	Week 3–4	PostgreSQL DDL, constraints, triggers, stored procedures; Neo4j graph loaded from DrugBank/FAERS with defined relationship types.
3. Application layer	Week 5–7	Next.js app: auth/roles, prescribing flow, "Analyze Risk" + path explanation view, admin dashboard.
4. Experiments & validation	Week 8–9	Experiments 1–3 (coverage, depth sensitivity, performance at scale) run and logged; Experiment 4 attempted if time allows.
5. Documentation & demo prep	Week 10	Full documentation per Section 14 of the course rubric; final demo script; README with setup/execution instructions.

10. Known Limitations (Stated Upfront)

- Patients are synthetic; results validate the system's traversal/explanation capability, not real clinical outcomes.
- DrugBank's mechanism-level (protein/enzyme) annotation coverage is uneven across drugs — graph depth and drug selection will be scoped to where documentation is strongest, rather than forcing 3-hop reasoning everywhere.
- This project does not compete with, or attempt to outperform, ML-based DDI prediction research (MolecBioNet, KnowDDI) — it addresses a different, narrower, systems-level question.

Prepared for instructor review — DBMS Experiential Learning, Level 3 Project. References available on request: DrugBank 6.0 (Knox et al., Nucleic Acids Research, 2024); MolecBioNet (Chen, Zhang & Qu, KDD 2025); KnowDDI (Wang, Yang & Yao, Nature Communications Medicine, 2024).